TECHNOLOGY STACK ANALYSIS

Developer: [Left Blank] | OptiCrop Project Documentation

Repository: https://github.com/goutham-05-raj/OptiCrop-Smart-Agricultural-Production-Optimization-Engine

1. Core Technology Selection

The technology stack for OptiCrop was selected to support fast execution, simple deployment, and data visualization:

Technology Component	Specific Choice	Role in Project
Programming Language	Python 3.10+	Data manipulation and model inference.
Web Framework	Flask 3.0.0	Maintains routes, handles forms, and renders template
Machine Learning	Scikit-learn	StandardScaler, RandomForest, and KMeans model p
Data Structures	Pandas & NumPy	Data frame cleaning, formatting, and numeric operatio
Graphics Engine	Matplotlib & Seaborn	Generates performance evaluation plots.
Frontend CSS	Bootstrap 5 & Vanilla CSS	Styles the dark-themed glassmorphic user dashboard.

Opticrop Technology Stack

Language : Python 3.10+
Framework : Flask 3.1.3 - routing, Jinja2 templates, REST endpoints
ML / AI : Scikit-learn 1.9 - Random Forest (classifier + regressor), K-Means
Data : Pandas 3.0, NumPy 2.5
Plotting : Matplotlib 3.11, Seaborn 0.13 - accuracy charts, confusion matrices
Frontend : HTML5, CSS3 (glassmorphism), Bootstrap 5
Testing : pytest / Python unittest
Version Ctrl: Git / GitHub

Opticrop Technology Stack

Language : Python 3.10+
Framework : Flask 3.1.3 - routing, Jinja2 templates, REST endpoints
ML / AI : Scikit-learn 1.9 - Random Forest (classifier + regressor), K-Means
Data : Pandas 3.0, NumPy 2.5
Plotting : Matplotlib 3.11, Seaborn 0.13 - accuracy charts, confusion matrices
Frontend : HTML5, CSS3 (glassmorphism), Bootstrap 5
Testing : pytest / Python unittest
Version Ctrl: Git / GitHub

2. Design Architecture Rationale

Flask was chosen as the web frame because it integrates easily with Python-based scikit-learn models. A local CSV and SQLite database storage layout was selected to avoid external dependencies, keeping the application lightweight and simple to run.